# Python OOP: Pure Inheritance

Focused practice problems covering Single, Multilevel, Multiple, and Hierarchical Inheritance. Free of Encapsulation, Polymorphism, or Abstraction.

SINGLE INHERITANCE | MULTILEVEL INHERITANCE | MULTIPLE INHERITANCE | HIERARCHICAL INHERITANCE

## Task 1: Person and Student

Single Inheritance

**PROBLEM STATEMENT**

Design a basic student registration model using single inheritance:

1. Create a parent class `Person` initializing with `first_name` and `last_name`. Add method `get_full_name()` returning the combined full name.
2. Create a child class `Student` inheriting from `Person` with an extra attribute `student_id`. Use `super()` in the constructor. Add method `get_student_info()` that calls `get_full_name()` and appends the ID.

**SAMPLE INPUT**

```
First Name: "Alice" Last Name: "Smith" ID: "ST1024" Actions: - get_student_info()
```

**SAMPLE OUTPUT**

```
Student: Alice Smith (ID: ST1024)
```

## Task 2: Vehicle to Electric Car

Multilevel Inheritance

**PROBLEM STATEMENT**

Design a 3-level vehicle classification hierarchy:

1. Top parent `Vehicle` initializes with `brand` and has method `start_engine()` returning `"Engine started"`.
2. Second-level `Car` inherits from `Vehicle`, adding `num_doors`.
3. Third-level `ElectricCar` inherits from `Car`, adding `battery_size`. Add method `get_full_specs()` combining details from all 3 levels.

**SAMPLE INPUT**

```
Brand: "Tesla", Doors: 4, Battery: 75 Actions: - get_full_specs()
```

**SAMPLE OUTPUT**

```
Tesla (4 doors) - 75 kWh Battery. Engine status: Engine started
```

## Task 3: Phone & Camera to Smartphone

Multiple Inheritance

**PROBLEM STATEMENT**

Design a device model inheriting from two separate parents:

1. Parent Phone takes phone_number and has method `make_call(target)` returning `"Calling [target]..."`.
2. Parent `Camera` takes `resolution` and has method `take_photo()` returning `"Photo taken in [resolution]"`.
3. Child `SmartPhone` inherits from both Phone and `Camera`, initializing attributes for both.

**SAMPLE INPUT**

```
Phone Number: "123-456-7890" Resolution: "108MP" Actions: - make_call("987-654-3210") - take_photo()
```

**SAMPLE OUTPUT**

```
Calling 987-654-3210... Photo taken in 108MP
```

## Task 4: Library Media Catalog

Hierarchical Inheritance

**PROBLEM STATEMENT**

Design a library system with two child classes inheriting from one parent:

1. Base class `LibraryItem` takes `title` and `item_id`. Method `get_base_info()` returns `"[title] (ID: [item_id])"`.
2. Child Book inherits from `LibraryItem`, adds `author`, and method `get_book_details()`.
3. Child `Magazine` inherits from `LibraryItem`, adds `issue_number`, and method `get_magazine_details()`.

**SAMPLE INPUT**

```
Book: Title="1984", ID="B01", Author="George Orwell" Magazine: Title="NatGeo", ID="M88", Issue=250 Actions: - book.get_book_details() - magazine.get_magazine_details()
```

**SAMPLE OUTPUT**

```
Book: 1984 (ID: B01) by George Orwell Magazine: NatGeo (ID: M88) - Issue #250
```

## Task 5: Rectangle and Box Volume

Single Inheritance

**PROBLEM STATEMENT**

Extend 2D shape features to 3D shapes using attribute inheritance:

1. Parent `Rectangle` initializes with `length` and `width`. Method `area()` returns `length * width`.
2. Child Box inherits from `Rectangle` and adds `height`, passing `length` and `width` to `super().__init__()`.
3. Add method `volume()` to Box calculating volume as `self.area() * self.height`.

**SAMPLE INPUT**

```
Length: 4, Width: 5, Height: 10 Actions: - area() - volume()
```

**SAMPLE OUTPUT**

```
Base Area: 20 Box Volume: 200
```

## Task 6: Employee and Manager Compensation

Single Inheritance

**PROBLEM STATEMENT**

Design a payroll system where child classes reuse parent math methods:

1. Base class `Employee` initializes with `name` and `base_salary`. Add method `calculate_annual_salary()` returning `base_salary * 12`.
2. Subclass `Manager` inherits from `Employee` and adds `monthly_bonus` using `super()` for initialization.
3. Add NEW method `calculate_total_annual_compensation()` to `Manager` that calls `self.calculate_annual_salary()` and adds `(monthly_bonus * 12)`.

**SAMPLE INPUT**

```
Name: "Robert", Base Salary: 5000, Bonus: 1000 Actions: - calculate_total_annual_compensation()
```

**SAMPLE OUTPUT**

```
72000
```

## Task 7: User to Admin Permissions

Multilevel Inheritance

**PROBLEM STATEMENT**

Design a 3-level account privilege model:

1. Base class `Account` initializes with `username`. Method `get_username()` returns username string.
2. Child class `User` inherits from `Account`, adding `email` and method `get_contact_info()`.
3. Child class `AdminUser` inherits from `User`, adding list `permissions`. Add method `show_admin_profile()` displaying details inherited across all 3 levels.

**SAMPLE INPUT**

```
Username: "admin_sarah" Email: "sarah@company.com" Permissions: ["create_db", "drop_db"] Actions: - show_admin_profile()
```

**SAMPLE OUTPUT**

```
Admin: admin_sarah | Email: sarah@company.com | Permissions: create_db, drop_db
```

## Task 8: Appliance & Energy Calculator

Single Inheritance

**PROBLEM STATEMENT**

Design a power consumption model using inherited attributes:

1. Parent `Appliance` takes `brand` and `wattage`. Method `get_wattage()` returns wattage.
2. Child `WashingMachine` inherits from `Appliance`, adding `capacity_kg` using `super()`.
3. Add method `calculate_daily_energy_kwh(hours_used)` calculating energy using inherited wattage: `(self.get_wattage() * hours_used) / 1000`.

**SAMPLE INPUT**

```
Brand: "LG", Wattage: 2000, Capacity: 7.5 kg Actions: - calculate_daily_energy_kwh(3)
```

**SAMPLE OUTPUT**

```
6.0 kWh
```

## Task 9: Writer & Speaker to AuthorPresenter

Multiple Inheritance

**PROBLEM STATEMENT**

Design a multi-trait aggregator class using multiple inheritance:

1. Class `Writer` takes `books_published` with method `get_writer_stats()`.
2. Class `Speaker` takes `keynotes_given` with method `get_speaker_stats()`.
3. Child `AuthorPresenter` inherits from both `Writer` and `Speaker`, adding a `name` attribute and method `get_full_portfolio()` combining outputs from both parents.

**SAMPLE INPUT**

```
Name: "Dr. Mark", Books: 5, Keynotes: 12 Actions: - get_full_portfolio()
```

**SAMPLE OUTPUT**

```
Portfolio for Dr. Mark: Published 5 books | Delivered 12 keynotes
```

## Task 10: Animal & Bird Hierarchy

Hierarchical Inheritance

**PROBLEM STATEMENT**

Design a shared species hierarchy without overriding parent behavior:

1. Base class `Animal` takes `species` and `age`. Method `get_basic_info()` returns `"[species] (Age: [age])"`.
2. Child `Bird` inherits from `Animal`, adding `wing_span` and method `fly_info()` that invokes `get_basic_info()`.
3. Child `Fish` inherits from `Animal`, adding `swimming_depth` and method `swim_info()` that invokes `get_basic_info()`.

**SAMPLE INPUT**

```
Bird: Species="Eagle", Age=4, Wingspan=2.1 Fish: Species="Salmon", Age=2, Depth=50 Actions: - bird.fly_info() - fish.swim_info()
```

**SAMPLE OUTPUT**

```
Eagle (Age: 4) with a wingspan of 2.1m Salmon (Age: 2) swims at depth 50m
```

### Implementation Constraints & Rules

- **Strict Inheritance Only:** Do not implement private attributes (`__attribute`) or getter/setter encapsulation.
- **No Method Overriding:** Do not override parent methods in child classes; call or extend parent methods directly using `super()` or parent attributes.
- **No Abstract Base Classes:** Do not use the `abc` module or standard abstract class patterns.
- **Constructor Chaining:** Always ensure `super().__init__(...)` or explicit parent calls are used to avoid attribute duplication.